

The topology of software risk in scientific models

2. Uncertainty and sensitivity analysis

Arnald Puy

Contents

1	Preliminary	2
2	The uncertainty and sensitivity analysis	3
3	Figures	4
4	Session information	13

1 Preliminary

```
# PRELIMINARY FUNCTIONS #####  
#####  
  
sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",  
                           "cowplot", "readxl", "ggrepel", "tidytext", "here",  
                           "tidygraph", "igraph", "foreach", "parallel", "ggraph",  
                           "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk",  
                           "ggribes"))  
  
# Set seed -----  
  
seed <- 123
```

2 The uncertainty and sensitivity analysis

```
# UNCERTAINTY AND SENSITIVITY ANALYSIS #####
#####

# Load the synthetic graph -----

data("synthetic_graph")

# Compute all paths -----

out <- all_paths_fun(graph = synthetic_graph, risk_form = "additive")
paths_dt <- data.table(out$paths)

# Define UA / SA settings -----

N <- 2^10
order <- "first"

# Run the UA / SA -----

output_ua <- softwareRisk::uncertainty_fun(all_paths_out = out, N = N,
                                           order = order, risk_form = "power_mean")

# CREATE THE DATASETS #####
#####

# Create datasetS -----

paths_ua_dt <- output_ua[["paths"]][
  , .(path_id, uncertainty_analysis, gini_index, risk_trend)]

for (nm in c("uncertainty_analysis", "risk_trend", "gini_index")) {

  prefix <- switch(nm, uncertainty_analysis = "P", risk_trend = "risk",
                  gini_index = "gini")

  paths_ua_dt[, c(paste0(prefix, "_median"),
                 paste0(prefix, "_q5"),
                 paste0(prefix, "_q95"))
  ] := .(
    vapply(get(nm), median, numeric(1)),
    vapply(get(nm), quantile, numeric(1), probs = 0.05),
    vapply(get(nm), quantile, numeric(1), probs = 0.95)
  )
}

# Create order to plot -----
```

```

path_order <- paths_ua_dt %>%
  .[order(-P_median)] %>%
  .[, path_id]

# Calculate mean risk per node -----

nodes_dt <- output_ua[["nodes"]] %>%
  .[, mean_risk:= vapply(uncertainty_analysis, mean, numeric(1))]

# Extract sensitivity analysis -----

nodes_dt_unnested <- nodes_dt[, {
  sa <- as.data.table(as.data.frame(sensitivity_analysis[[1]]$results))
  cbind(.SD[, !"sensitivity_analysis"], sa)
},
  by = name
]

```

3 Figures

```

# PLOT ERRORBARS #####

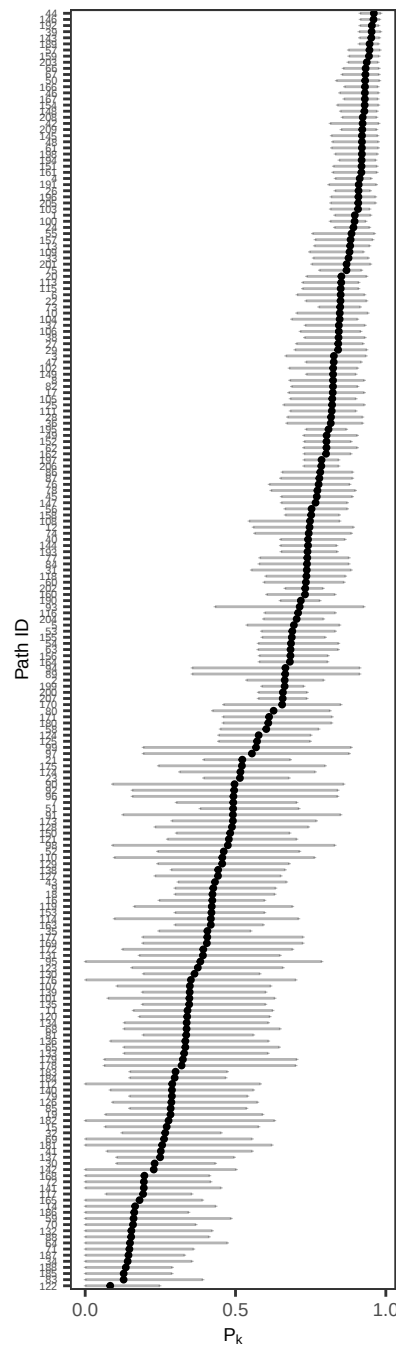
# Plot risk slope -----

paths_ua_dt[, path_id:= factor(path_id, levels = rev(path_order))]

a <- paths_ua_dt %>%
  ggplot(., aes(P_median, path_id)) +
  geom_point(size = 0.7) +
  geom_errorbar(aes(xmin = P_q5, xmax = P_q95), height = 0.2, alpha = 0.3) +
  labs(y = "Path ID", x = expression(P[k])) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
  theme(axis.text.y = element_text(size = 4),
        legend.position = c(0.4, 0.87),
        plot.margin = margin(1, 0.1, 1, 1))
a

```

`height` was translated to `width`.

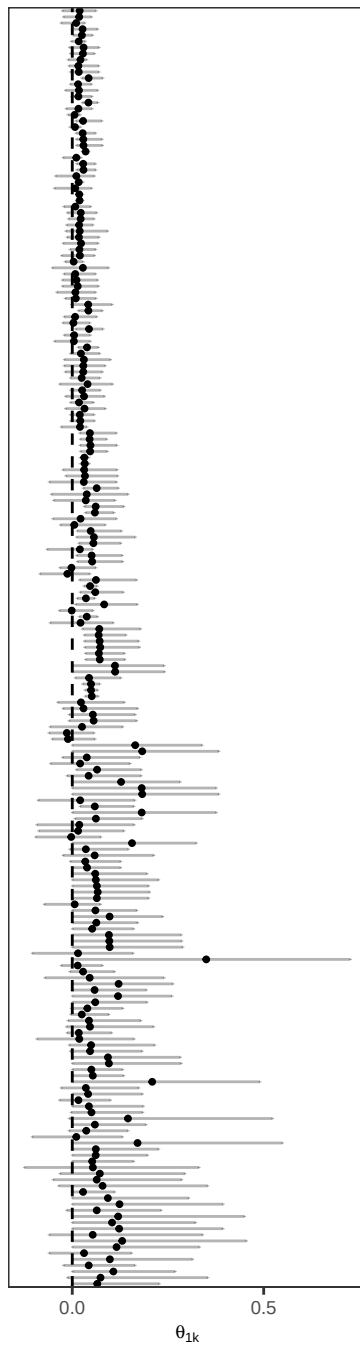


```
# Plot risk slope -----
b <- paths_ua_dt %>%
  ggplot(., aes(risk_median, path_id)) +
  geom_point(size = 0.7) +
  geom_vline(xintercept = 0, lty = 2) +
  labs(x = expression(theta[1*k]), y = "") +
  geom_errorbar(aes(xmin = risk_q5, xmax = risk_q95), height = 0.2, alpha = 0.3) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
```

```
theme(legend.position = "none",  
      axis.text.y = element_blank(),  
      axis.ticks.y = element_blank(),  
      plot.margin = margin(1, 0.1, 1, 0.1))
```

b

`height` was translated to `width`.

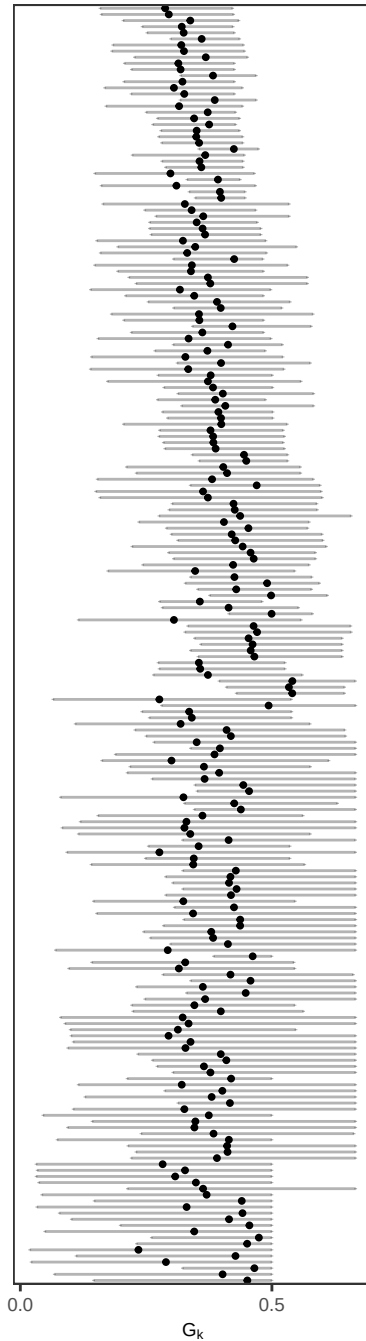


```
# Plot Gini index -----

c <- paths_ua_dt %>%
  ggplot(., aes(gini_median, path_id)) +
  geom_point(size = 0.7) +
  labs(x = expression(G[k]), y = "") +
  geom_errorbar(aes(xmin = gini_q5, xmax = gini_q95), height = 0.2, alpha = 0.3) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "none",
        axis.text.y = element_blank(),
        axis.ticks.y = element_blank(),
        plot.margin = margin(1, 0.1, 1, 0.1))

c
```

`height` was translated to `width`.



```
# PLOT SENSITIVITY ANALYSIS #####

dt_plot <- nodes_dt_unnested %>%
  .[, .(name, original, sensitivity, parameters)] %>%
  .[sensitivity != "Ti"] %>%
  .[, name:= factor(name, levels = unique(name[order(-original)]))]

plot_sa <- ggplot(dt_plot, aes(x = parameters, y = name, fill = original)) +
  geom_tile() +
  scale_fill_viridis_c(name = expression(S[p]), breaks = c(0, 0.3, 0.6)) +
```

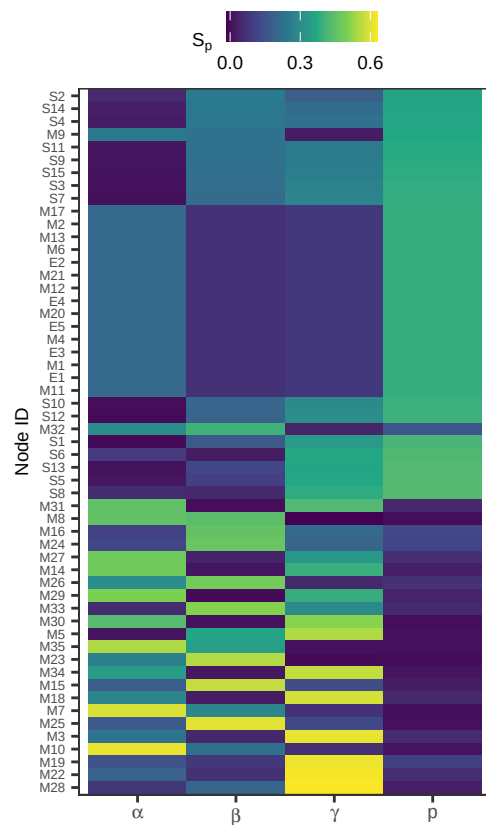


```

theme_AP() +
scale_x_discrete(labels = c("a_raw" = expression(alpha),
                             "b_raw" = expression(beta),
                             "c_raw" = expression(gamma),
                             "p" = expression(p))) +
labs(x = "", y = "Node ID") +
theme(axis.text.y = element_text(size = 5),
      legend.position = "top",
      plot.margin = margin(1, 0.1, 1, 1))

plot_sa

```



```

# BUMP CHART PLOT #####

out_compensatory <- all_paths_fun(graph = synthetic_graph,
                                risk_form = "power_mean", p = 0.5)

out_bottleneck <- all_paths_fun(graph = synthetic_graph,
                                risk_form = "power_mean", p = 2)

paths_dt <- rbind(out_compensatory$paths %>%
                  data.table() %>%
                    .[, aggregation:= "compensatory"],
                  out_bottleneck$paths %>%

```

```

data.table() %>%
  .[, aggregation:= "bottleneck"]]

# Prepare dataset and calculate mean ranks -----

paths_dt[, rank:= frank(-path_risk_score, ties.method = "average"), aggregation]

# keep top 20 per functional form -----

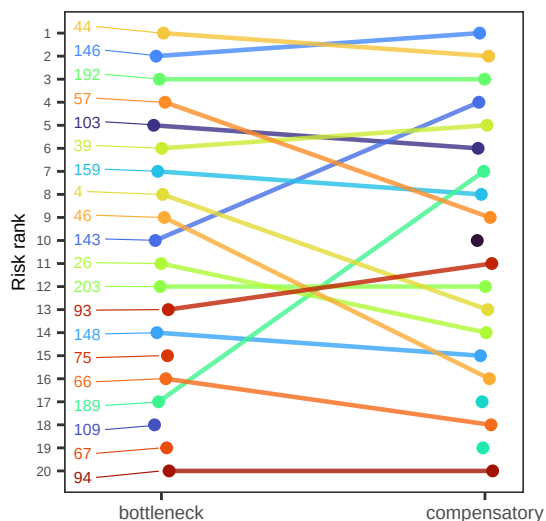
top20 <- paths_dt[rank <= 20]

# Path_id should be character -----
top20[, path_id_chr := as.character(path_id)]

bump_chart_plot <- ggplot(top20, aes(aggregation, rank, group = path_id_chr,
                                     colour = path_id_chr)) +
  geom_line(linewidth = 0.8, alpha = 0.8, position = position_dodge(width = 0.05)) +
  geom_point(size = 1.6, position = position_dodge(width = 0.05)) +
  geom_text_repel(data = top20[aggregation == "bottleneck"],
                 aes(label = path_id_chr, colour = path_id_chr),
                 size = 2, nudge_x = -0.3, direction = "y",
                 segment.size = 0.2, show.legend = FALSE) +
  scale_y_reverse(breaks = 1:20) +
  scale_x_discrete(expand = expansion(mult = c(0.2, 0.12))) +
  scale_colour_viridis_d(option = "turbo", end = 0.95) +
  theme_AP() +
  theme(legend.position = "none",
        plot.margin = margin(12, 12, 12, 12),
        axis.text.y = element_text(size = 5)) +
  labs(y = "Risk rank", x = NULL)

```

bump_chart_plot



```

# MERGE UA / SA PLOTS #####

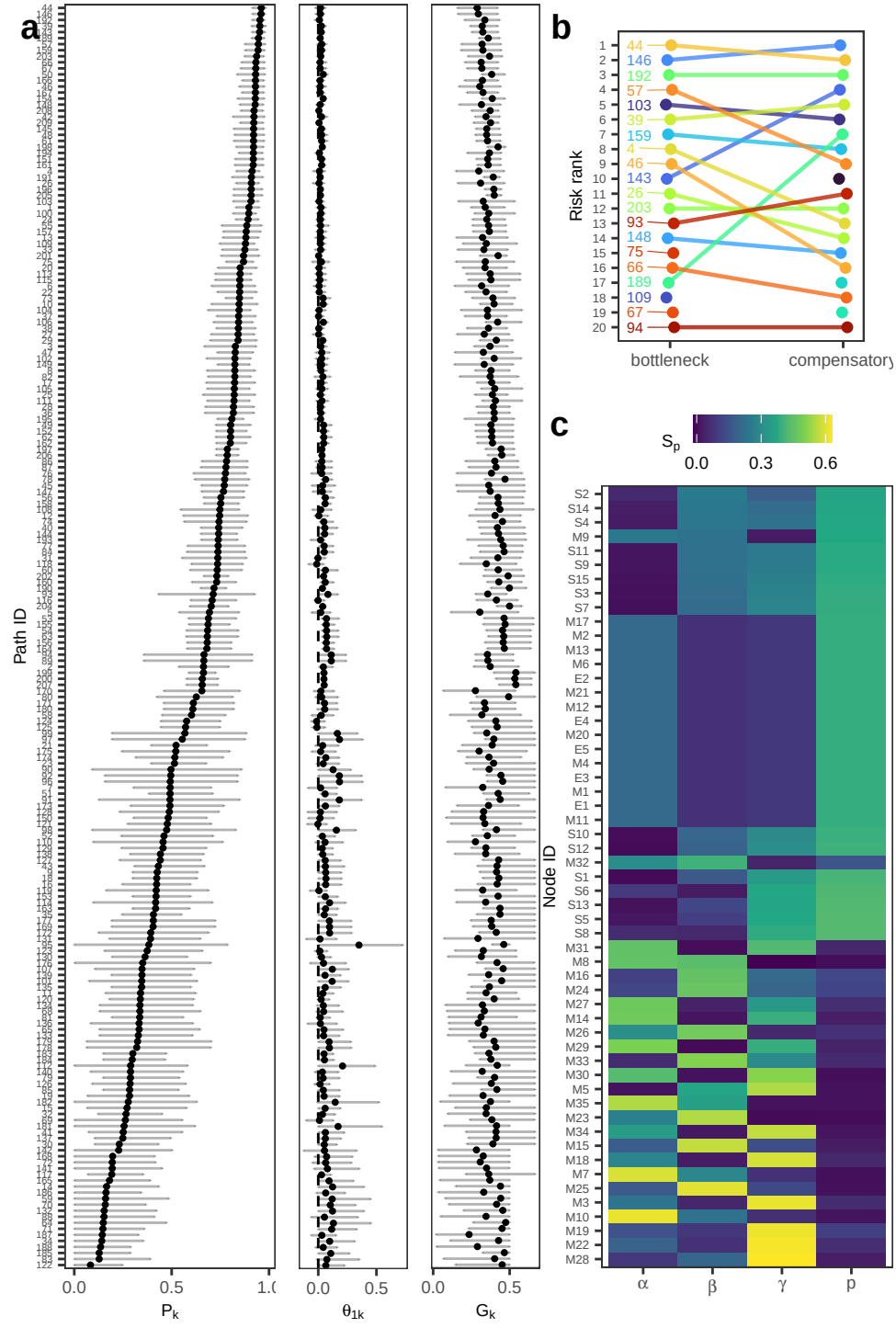
left <- plot_grid(a, b, c, ncol = 3, rel_widths = c(0.5, 0.25, 0.25), labels = c("a", "", ""))

## `height` was translated to `width`.
## `height` was translated to `width`.
## `height` was translated to `width`.

right <- plot_grid(bump_chart_plot, plot_sa, ncol = 1, rel_heights = c(0.3, 0.7),
  labels = c("b", "c"))

plot_grid(left, right, ncol = 2, rel_widths = c(0.6, 0.4))

```



4 Session information

```
# SESSION INFORMATION #####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] tools parallel stats graphics grDevices utils datasets
## [8] methods base
##
## other attached packages:
## [1] ggribges_0.5.7 softwareRisk_0.1.0 benchmarkme_1.0.8 sensobol_1.1.6
## [5] ggraph_2.2.2 foreach_1.5.2 igraph_2.2.1 tidygraph_1.3.1
## [9] here_1.0.2 tidytext_0.4.3 ggrepel_0.9.6 readxl_1.4.5
## [13] cowplot_1.2.0.9000 scales_1.4.0 openxlsx_4.2.8 lubridate_1.9.4
## [17] forcats_1.0.1 stringr_1.6.0 dplyr_1.1.4 purrr_1.2.1
## [21] readr_2.1.5 tidyr_1.3.2 tibble_3.3.1 ggplot2_4.0.1.9000
## [25] tidyverse_2.0.0 data.table_1.18.0
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1 viridisLite_0.4.2 farver_2.1.2
## [4] viridis_0.6.5 S7_0.2.1 fastmap_1.2.0
## [7] tweenr_2.0.3 janeaustenr_1.0.0 digest_0.6.39
## [10] timechange_0.3.0 lifecycle_1.0.5 tokenizers_0.3.0
## [13] magrittr_2.0.4 compiler_4.5.2 rlang_1.1.7
## [16] yaml_2.3.12 knitr_1.51 labeling_0.4.3
## [19] graphlayouts_1.2.2 RColorBrewer_1.1-3 withr_3.0.2
## [22] grid_4.5.2 polyclip_1.10-7 iterators_1.0.14
## [25] MASS_7.3-65 tinytex_0.58 cli_3.6.5
## [28] rmarkdown_2.30 generics_0.1.4 rstudioapi_0.17.1
## [31] http_1.4.7 tzdb_0.5.0 cachem_1.1.0
## [34] ggforce_0.5.0 cellranger_1.1.0 vctrs_0.6.5
## [37] Matrix_1.7-4 hms_1.1.3 ineq_0.2-13
## [40] glue_1.8.0 benchmarkmeData_1.0.4 codetools_0.2-20
```

```
## [43] stringi_1.8.7      gtable_0.3.6      pillar_1.11.1
## [46] htmltools_0.5.9    R6_2.6.1          Rdpack_2.6.4
## [49] doParallel_1.0.17  rprojroot_2.1.1   evaluate_1.0.5
## [52] lattice_0.22-7     rbibutils_2.4     SnowballC_0.7.1
## [55] memoise_2.0.1      Rcpp_1.1.1        zip_2.3.3
## [58] gridExtra_2.3      xfun_0.55         pkgconfig_2.0.3
```

```
## Return the machine CPU -----
```

```
cat("Machine:    "); print(get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
## Return number of true cores -----
```

```
cat("Num cores:   "); print(detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
## Return number of threads -----
```

```
cat("Num threads: "); print(detectCores(logical = FALSE))
```

```
## Num threads:
```

```
## [1] 10
```